МИНИСТЕРСТВО ОБРАЗОВАНИЯ СВЕРДЛОВСКОЙ ОБЛАСТИ
ГАПОУ СО «ЕКАТЕРИНБУРГСКИЙ МОНТАЖНЫЙ КОЛЛЕДЖ»

КУРСОВОЙ ПРОЕКТ

По дисциплине «МДК 05.03 Тестирование информационных систем»

ТЕМА: «Создание и тестирование АРМ застройщика»

ЕМК 09.02.07 КП 7050ИП-31к-25

ОЦЕНКА: _____

РУКОВОДИТЕЛЬ КУРСОВОГО ПРОЕКТА:
_____( )

СТУДЕНТ гр. ИП-31/ИП-31к _____(Твердов Д.С.)

2026

Ведомость курсового проекта

№ стр.	Формат	Обозначение	Наименование	Кол. листов	№ экз.	Примеч.
1	А4	ЕМК 09.02.07 КП 7050ИП-31к-25 ПЗ	Пояснительная записка	48	-	
2		ЕМК 09.02.07 КП 7050ИП-31к-25 ИС	«Создание и тестирование АРМ застройщика»	1		USB флеш-карта

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ 3
ВВЕДЕНИЕ 5
1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ 6
2. АНАЛИТИЧЕСКАЯ ЧАСТЬ 7
2.1. Цели, функции и задачи системы 7
2.2. Описание предметной области системы 9
2.3. Определение пользователей системы 11
2.4. Проектные решения по реализации внутренней архитектуры приложения 13
2.5. Проектные решения по реализации тестирования приложения 15
3. ПРОЕКТНАЯ ЧАСТЬ 17
3.1. Стек технологий 17
3.2. ERD-модель информационной системы 19
3.3. Файловая структура приложения 26
3.4. Алгоритмы 30
3.5. Документация по работе с информационной системой 33
3.6. Отчёт о тестировании проекта 41
4. ГЛОССАРИЙ 44
5. СПИСОК ИСТОЧНИКОВ 45
ЗАКЛЮЧЕНИЕ 46
ПРИЛОЖЕНИЕ А 47
ПРИЛОЖЕНИЕ Б 49

ВВЕДЕНИЕ

В современных условиях развития рынка недвижимости автоматизация процессов учёта и управления объектами строительства становится важной задачей для застройщиков. Ручное ведение данных об объектах недвижимости, фиксация продаж, формирование договоров купли-продажи и отслеживание статусов сделок являются трудоёмкими процессами, подверженными ошибкам и не обеспечивающими необходимой оперативности. В связи с этим разработка автоматизированного рабочего места (АРМ) застройщика является актуальной задачей, направленной на повышение эффективности управления объектами недвижимости и качества обслуживания клиентов.

Целью курсового проекта является разработка и тестирование информационной системы для автоматизации хранения и учёта информации об объектах недвижимости, управления фотографиями и строительной документацией, ведения статусов объектов и формирования договоров купли-продажи. Для достижения поставленной цели необходимо провести анализ предметной области и сформулировать требования на основе технического задания, разработать архитектуру приложения, обеспечивающую масштабируемость, надёжность и безопасность, реализовать ролевую модель доступа для различных категорий пользователей, а также выполнить тестирование разработанной системы, включая функциональное, интеграционное и UI-тестирование.

Объектом исследования является процесс автоматизации деятельности застройщика по учёту и управлению объектами недвижимости. Предметом исследования выступают методы и технологии проектирования, разработки и тестирования клиент-серверных информационных систем. Разрабатываемая система предназначена для централизованного хранения и обработки данных об объектах недвижимости, их фотографиях, строительной документации, договорах купли-продажи, а также для аудита действий пользователей.

Практическая значимость работы заключается в создании информационной системы, способной повысить эффективность работы застройщика, упростить обработку данных об объектах недвижимости, автоматизировать формирование договоров купли-продажи и обеспечить прозрачность учёта имущества.

1. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Тема курсового проекта: «Создание и тестирование АРМ застройщика»

Выдано Твердову Даниилу Сергеевичу студенту группы ИП-31/ИП-31к

Сроки выполнения:

1. разработка REST-сервиса с использованием фреймворка FastAPI: до 15.12.25г
2. разработка web-приложения на Django: до 1.05.26г

ВИД АВТОМАТИЗИРУЕМОЙ ДЕЯТЕЛЬНОСТИ

Необходимо разработать приложение, предназначенное для автоматизации хранения и учёта информации об объектах недвижимости.

Основными функциями автоматизированного рабочего места являются:

- добавление, редактирование информации о продаваемом объекте недвижимости;
- добавление фото строительного объекта недвижимости (возможно прикрепление нескольких фото);
- изменение фото строительного объекта недвижимости;
- удаление данных об объекте недвижимости;
- поиск информации об объектах недвижимости: поиск определяется видом недвижимого имущества;
- добавление строительной документации по объекту (схема, план застройки);
- удаление строительной документации по объекту недвижимости;
- просмотр всех имеющихся объектов недвижимости;
- установление статуса объекту недвижимости (продан, продажа и т.п.);
- изменение статуса объекта недвижимости (продан, продажа);
- задавать не только основную цену, но и скидочную цену объектов недвижимости;
- распечатывать договор купли-продажи объекта недвижимости (Word/Excel).

По каждому объекту недвижимости заносится:

- Тип объекта;
- Общая площадь;
- Отделка;
- Материал стен;
- Этаж;
- Цена;

– Фотографии, схемы;
– И т.п.

Пользователи информационной системы:
– незарегистрированные пользователи;
– зарегистрированные пользователи;
– менеджеры;
– администраторы.

Незарегистрированные пользователи могут только просматривать информацию о строительных объектах и строительной документации.

Зарегистрированные пользователи могут также просматривать информацию о договорах с застройщиком.

Администраторы ведут сопровождение (заполнение, редактирование информации о строительных объектах и объектах недвижимости).

Менеджеры ведут фиксацию продажи объектов недвижимости.

2. АНАЛИТИЧЕСКАЯ ЧАСТЬ

2.1. Цели, функции и задачи системы

Актуальность разработки автоматизированного рабочего места застройщика обусловлена необходимостью повышения эффективности управления объектами недвижимости в условиях постоянного роста рынка строительства и увеличения объёма данных о продаваемых объектах. В процессе работы застройщика ежедневно выполняются операции, связанные с регистрацией новых объектов недвижимости, ведением справочника типов объектов, оформлением договоров купли-продажи, изменением статусов сделок и контролем доступа пользователей к различным функциям системы. При использовании ручного учёта или разрозненных программных решений возрастает вероятность ошибок, усложняется контроль данных, теряется прозрачность действий и увеличивается нагрузка на сотрудников.

Основной целью разрабатываемой системы является создание единого программного комплекса, который позволит автоматизировать основные процессы учёта объектов недвижимости, обеспечить централизованное хранение информации и повысить удобство взаимодействия между администраторами, менеджерами, зарегистрированными и незарегистрированными пользователями. Предполагается, что система будет обеспечивать безопасную обработку данных, поддержку ролевой модели доступа и возможность дальнейшего расширения функциональности.

Информационная система будет включать несколько основных компонентов. Одним из компонентов станет API-бэкенд с поддержкой версионирования API (v1 и v2). API будет использоваться для обработки запросов, взаимодействия с базой данных, реализации бизнес-логики и поддержки механизмов аутентификации и авторизации пользователей. Для API предполагается использование автоматической документации Swagger/OpenAPI. Вторым компонентом системы станет полнофункциональное веб-приложение, реализуемое на основе Django. Веб-приложение будет использовать шаблоны, формы и административную панель для взаимодействия с пользователями. Получение и обработка данных будут выполняться через REST API, что позволит разделить пользовательский интерфейс и серверную бизнес-логику. Для хранения информации предполагается использование клиент-серверной базы данных PostgreSQL, обеспечивающей централизованное хранение данных об объектах недвижимости, пользователях, договорах купли-продажи, фотографиях, строительной документации и журнале аудита.

К основным функциям системы относятся регистрация и авторизация пользователей, управление ролями и правами доступа (администратор, менеджер,

зарегистрированный пользователь, внешний пользователь), хранение и обработка данных об объектах недвижимости и их характеристиках, управление фотографиями и строительной документацией объектов, учёт договоров купли-продажи с возможностью формирования документов Word/Excel, ведение журнала аудита действий пользователей, а также администрирование системы (управление пользователями и справочниками).

Для достижения поставленной цели необходимо решить следующие задачи: выполнить анализ предметной области и определить требования к информационной системе учёта объектов недвижимости, разработать архитектуру приложения и структуру взаимодействия между веб-приложением, API и базой данных, реализовать механизмы аутентификации, авторизации и разграничения прав доступа, разработать основные модули системы и REST API, а также выполнить тестирование основных функций и компонентов приложения.

2.2. Описание предметной области системы

Предметная область проекта охватывает деятельность застройщика в части учёта объектов недвижимости, управления их фотографиями и строительной документацией, ведения договоров купли-продажи и контроля взаимодействия между пользователями системы. Анализ предметной области выполняется не только как описание сущностей базы данных, но и как исследование основных процессов работы застройщика, требующих автоматизации.

В основу проектирования системы предполагается положить последовательность основных этапов взаимодействия пользователей: регистрация объекта → добавление фотографий и документации → установка статуса и цены → оформление договора купли-продажи → аудит действий. Именно данный процесс определяет состав сущностей и взаимосвязей внутри системы. Основная задача проектирования заключается в обеспечении непрерывности хранения и обработки данных на всех этапах работы пользователей с приложением.

С точки зрения информационных потоков в системе предполагается выделение следующих основных объектов предметной области:

- объект недвижимости (тип, площадь, отделка, материал стен, этаж, основная и скидочная цены, статус, адрес, описание);
- тип объекта недвижимости (квартира, дом, коммерческая площадь, и т.д.);
- фотография объекта (файл, описание, дата загрузки, ссылка на объект);
- строительная документация (схема, план застройки, наименование, тип, дата добавления);

- договор купли-продажи (данные покупателя и продавца, стоимость, дата, статус, примечания);
- пользователь системы (учётные данные, роль и права доступа);
- журнал аудита (действия пользователей, дата и время, тип операции).

Выделение именно этих сущностей обусловлено спецификой работы застройщика и необходимостью централизованного хранения информации об объектах недвижимости и фактах совершения сделок.

В предметной области предполагается использование нескольких ролей пользователей:

- администратор — полное управление системой, пользователями и объектами недвижимости;
- менеджер — фиксация продажи объектов недвижимости, управление договорами;
- зарегистрированный пользователь — просмотр информации о договорах с застройщиком;
- незарегистрированный пользователь — ограниченный доступ к общедоступной информации об объектах и строительной документации.

Такое разделение ролей позволит ограничить доступ пользователей только к тем функциям и данным, которые относятся к их обязанностям. Для реализации данного механизма предполагается использование ролевой модели доступа (RBAC) на основе JWT-токенов.

Для формирования структуры проекта предполагается изучение существующих информационных систем управления недвижимостью, типовых CRM-систем для застройщиков и порталов объявлений о продаже недвижимости. Также будут анализироваться процессы взаимодействия между сотрудниками и клиентами, порядок оформления договоров купли-продажи и принципы хранения данных об объектах. Ключевым критерием отбора исходных данных является их практическая значимость для работы системы.

При отсутствии единой информационной системы в работе застройщика возникают типовые проблемы:

- разрозненное хранение информации об объектах и документации;
- сложность отслеживания статуса сделок и изменения цен;
- необходимость повторного ручного ввода данных при формировании договоров;
- ошибки при регистрации объектов и учёте покупателей;
- отсутствие централизованного аудита действий сотрудников.

Автоматизация данных процессов позволит сократить количество рутинных операций и повысить эффективность работы застройщика.

2.3. Определение пользователей системы

Целевой аудиторией системы являются администраторы застройщика, менеджеры по продажам, зарегистрированные пользователи (потенциальные покупатели) и незарегистрированные пользователи, так как именно эти категории ежедневно взаимодействуют с данными об объектах недвижимости, договорах и документации. Интерфейс системы должен быть адаптирован не под технических специалистов, а под пользователей, которым необходимо быстро выполнять рабочие операции и получать доступ к нужной информации без сложной навигации.

Основной сценарий использования системы строится по принципу минимального количества действий: авторизация → выбор раздела → работа с объектами или договорами → сохранение изменений → просмотр результатов. Навигация системы должна быть организована по логике работы пользователей, а не по внутренней структуре модулей приложения.

Для системы предполагается использование ролевого интерфейса (role-based UI). Каждый пользователь будет видеть только те разделы и функции, которые относятся к его роли:

- администратор — полное управление объектами недвижимости, фотографиями, строительной документацией, статусами, ценами, договорами и пользователями;
- менеджер — фиксация продаж, управление договорами купли-продажи и изменение статусов объектов;
- зарегистрированный пользователь — просмотр своих договоров с застройщиком и информации об объектах;
- незарегистрированный пользователь — просмотр общедоступной информации об объектах недвижимости и строительной документации без персональных данных.

В рамках проектирования предлагается разработка UML-диаграммы прецедентов. На диаграмме будут отражены основные участники системы: администратор, менеджер, зарегистрированный и незарегистрированный пользователь, а также варианты их взаимодействия с системой.

Рисунок 1 — Диаграмма прецедентов

2.4. Проектные решения по реализации внутренней архитектуры приложения

В проекте планируется использование двухслойной архитектуры: Django применяется как web-слой пользовательского взаимодействия, а FastAPI — как сервисный

слой бизнес-логики и REST API. Такое решение позволяет разделить ответственность между интерфейсной частью и обработкой данных: web-слой отвечает за отображение страниц, формы, маршрутизацию и взаимодействие с пользователем, а API-слой — за обработку бизнес-операций, проверку ролей, валидацию данных и работу с базой данных.

Взаимодействие компонентов системы организуется по следующей схеме: пользователь → Django web-приложение → FastAPI REST API → PostgreSQL. Внутренняя архитектура проекта строится на основе модульного подхода. Функциональность системы разделяется на отдельные уровни и модули, каждый из которых отвечает за строго определённую область задач.

Архитектурные уровни приложения включают слой представления, слой бизнес-логики, слой доступа к данным и слой базы данных.

Слой представления (Presentation Layer) отвечает за взаимодействие пользователя с системой и обработку HTTP-запросов. В его состав входят FastAPI-роутеры, Django views и HTML-шаблоны, формы ввода данных, маршруты приложения, а также Pydantic-схемы для проверки входящих данных.

Слой бизнес-логики (Business Logic Layer) отвечает за реализацию основных правил работы системы и обработку пользовательских сценариев. В данном слое располагаются контроллеры, сервисы, механизмы проверки бизнес-правил и обработка ролевого доступа. Сервисный слой отвечает за добавление и редактирование объектов недвижимости, управление фотографиями и документацией, оформление договоров купли-продажи, ведение журнала аудита действий пользователей и проверку уникальности данных.

Слой доступа к данным (Data Access Layer) отвечает за взаимодействие приложения с базой данных PostgreSQL. В его состав входят ORM-модели, сессии базы данных и репозитории для работы с сущностями.

Слой базы данных (Database Layer) обеспечивает хранение информации обо всех сущностях системы. В PostgreSQL предполагается хранение учётных записей пользователей, данных об объектах недвижимости и их характеристиках, фотографий, строительной документации, договоров купли-продажи и журнала аудита действий.

2.5. Проектные решения по реализации тестирования приложения

Тестирование информационной системы АРМ застройщика рассматривается как отдельный этап контроля качества, направленный на подтверждение корректности работы пользовательских сценариев, API-маршрутов и механизмов разграничения доступа. Проверке подлежат как отдельные компоненты системы, так и взаимодействие между Django-интерфейсом и FastAPI-бэкендом.

Основными целями тестирования являются:

- проверка корректности работы пользовательских сценариев;
- подтверждение работоспособности REST API;
- проверка обработки JWT-токенов и ролевой модели доступа;
- выявление ошибок валидации входных данных;
- контроль корректности HTTP-статусов и сообщений об ошибках;
- проверка взаимодействия веб-интерфейса и API;
- обеспечение стабильности системы при изменении функциональности.

Для проекта используется комбинированная стратегия тестирования, включающая функциональное, интеграционное, UI-, API-, негативное и регрессионное тестирование.

Таблица 1 — План тестирования

Test Case ID	Объект тестирования	Стратегия тестирования	Критерии начала	Критерии окончания	Последовательность выполнения
TC_1	Регистрация и авторизация пользователей	UI, API, функциональное, негативное	Доступна форма регистрации и endpoint авторизации	JWT-токен выдается корректно	Регистрация → авторизация → проверка доступа
TC_2	CRUD-операции с объектами недвижимости	UI + API, функциональное	Подготовлены тестовые данные	Операции Create/Read/Update/Delete выполняются корректно	Создание → просмотр → редактирование → удаление
TC_3	Управление фотографиями объекта	Функциональное, регрессионное	Доступна роль администратора	Операции с фото выполняются без ошибок	Добавление фото → изменение → удаление
TC_4	Управление строительной документацией	Интеграционное, API	Созданы тестовые объекты	Документация сохраняется корректно	Добавление → сохранение → проверка БД
TC_5	Управление статусами и ценами	UI, функциональное	Активны объекты недвижимости	Статусы и цены изменяются корректно	Установка статуса → смена → проверка
TC_6	Формирование договора купли-продажи Word/Excel	Функциональное, интеграционное	Доступны данные объекта и покупателя	Договор формируется корректно	Формирование → экспорт файла
TC_7	Разграничение прав доступа	Негативное, API	Созданы роли пользователей	Доступ ограничивается корректно	Попытка доступа → проверка отказа

TC_8	Обработка невалидных запросов	Негативное тестирование	API доступен	Ошибки обрабатываются корректно	Отправка некорректных данных → проверка ошибок

Таблица 2 — Traceability Matrix

Requirement ID	Описание требования	Test Case ID	Статус покрытия
REQ_1	Регистрация пользователей	TC_1	Покрыто
REQ_2	Авторизация и JWT	TC_1	Покрыто
REQ_3	CRUD объектов недвижимости	TC_2	Покрыто
REQ_4	Управление фотографиями	TC_3	Покрыто
REQ_5	Строительная документация	TC_4	Покрыто
REQ_6	Статусы и цены объектов	TC_5	Покрыто
REQ_7	Экспорт договоров Word/Excel	TC_6	Покрыто
REQ_8	RBAC и разграничение доступа	TC_7	Покрыто
REQ_9	Обработка ошибок и невалидных запросов	TC_8	Покрыто

Для обеспечения качества разработки предполагается использование подхода Test-Driven Development (TDD). Разработка основных модулей системы выполняется через предварительное написание тестов, что позволяет контролировать корректность реализации функциональности и уменьшить вероятность появления ошибок.

3. ПРОЕКТНАЯ ЧАСТЬ

3.1. Стек технологий

Backend (основной язык и фреймворки):

- Python 3.11+ — основной язык разработки всех компонентов системы.
- FastAPI — асинхронный веб-фреймворк для реализации REST API (бизнес-логика, обработка запросов, автоматическая документация Swagger/ReDoc).
- Django 5.x — веб-фреймворк для реализации пользовательского интерфейса (шаблоны, формы, маршрутизация, административная панель). Django выступает клиентом к FastAPI.
- SQLAlchemy (через SQLModel) — ORM для работы с базой данных в FastAPI-части.
- Pydantic — валидация данных и сериализация (используется в FastAPI для схем запросов и ответов).

База данных:

- PostgreSQL 17+ — реляционная клиент-серверная СУБД для централизованного хранения данных.
- psycopg2 — драйвер для подключения к PostgreSQL.
- Alembic — инструмент для управления миграциями схемы базы данных (для FastAPI-части).
- Django ORM — используется в веб-приложении для работы с базой данных.

Аутентификация и безопасность:

- JWT (JSON Web Tokens) — механизм аутентификации и передачи прав доступа.
- python-jose — библиотека для создания и проверки JWT-токенов.
- passlib (bcrypt) — хеширование паролей пользователей.
- OAuth2 (Password Flow) — протокол авторизации, встроенный в FastAPI.

Тестовая инфраструктура:

- pytest — основной фреймворк для написания и запуска тестов.
- pytest-django — интеграция pytest с Django для тестирования представлений, форм и маршрутов.
- pytest-asyncio — поддержка асинхронных тестов для FastAPI.
- httpx — асинхронный HTTP-клиент для тестирования API-эндпоинтов.
- pytest-cov — измерение покрытия кода тестами.

– pytest-mock — mocking зависимостей в тестах.
– Selenium — UI-тестирование веб-интерфейса (пользовательские сценарии).

Дополнительные библиотеки:

– python-dotenv — управление переменными окружения.
– email-validator — валидация адресов электронной почты.
– pydantic-settings — централизованное управление настройками приложения.
– requests — взаимодействие Django-приложения с FastAPI (HTTP-запросы).
– openpyxl / xlsxwriter — генерация договоров и отчётов в формате Excel.
– python-docx — генерация договоров купли-продажи в формате Word.
– Pillow — обработка и валидация изображений при загрузке фотографий объектов.

3.2. ERD-модель информационной системы

Физическая модель базы данных представлена в виде ERD-диаграммы (Entity-Relationship Diagram), реализованной средствами СУБД PostgreSQL. Диаграмма отражает структуру таблиц, их атрибуты, первичные и внешние ключи, а также связи между сущностями.

Рисунок 2 — Схема базы данных

База данных информационной системы включает 7 взаимосвязанных таблиц: users, property_types, properties, photos, construction_docs, contracts, audit_logs. Таблица users хранит данные об операторах системы и администраторах. Таблица property_types содержит справочник типов объектов недвижимости. Таблица properties является центральной таблицей и содержит сведения о продаваемых объектах недвижимости. Таблица photos хранит фотографии объектов. Таблица construction_docs содержит строительную документацию по объектам. Таблица contracts хранит информацию о заключённых договорах купли-продажи. Таблица audit_logs обеспечивает ведение журнала всех действий пользователей в системе.

Таблица 3 — Классификация связей информационной системы

Родительская таблица	Дочерняя таблица	Тип связи	Обоснование
users	audit_logs	1:M	Один пользователь может совершить множество действий, каждое из которых фиксируется в журнале аудита

users	contracts	1:M	Один менеджер может оформить множество договоров купли-продажи
property_types	properties	1:M	Один тип объекта может быть присвоен множеству объектов недвижимости
properties	photos	1:M	Один объект недвижимости может иметь множество фотографий
properties	construction_docs	1:M	Один объект может содержать множество документов строительной документации
properties	contracts	1:M	Один объект недвижимости может фигурировать в нескольких договорах (предварительный, основной)

Словарь данных сформирован для однозначного понимания структуры и назначения каждого поля.

Таблица 4 — Таблица users (Пользователи)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор пользователя
	username	VARCHAR(100)	N	Уникальное	Имя пользователя для входа
	email	VARCHAR(100)	N	Уникальное	Адрес электронной почты
	full_name	VARCHAR(200)	Y		Полное имя пользователя
	hashed_password	TEXT	N		Хэш пароля пользователя
	role	VARCHAR(20)	N	DEFAULT 'user'	Роль пользователя (admin, manager, user, external)
	is_active	BOOLEAN	N	DEFAULT TRUE	Признак активности учётной записи
	created_at	TIMESTAMP WITH TIME ZONE	Y	DEFAULT NOW()	Дата и время регистрации
	updated_at	TIMESTAMP WITH TIME ZONE	Y	DEFAULT NOW()	Дата и время последнего обновления

Таблица 5 — Таблица property_types (Типы объектов)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор типа
	name	VARCHAR(100)	N	Уникальное	Наименование типа объекта
	description	TEXT	Y		Описание типа объекта недвижимости
	created_at	TIMESTAMP WITH TIME ZONE	Y	DEFAULT NOW()	Дата и время создания записи

Таблица 6 — Таблица properties (Объекты недвижимости)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор объекта
FK	type_id	INTEGER	N		ID типа объекта (ссылка на property_types.id)
	area	NUMERIC(10,2)	N		Общая площадь объекта (кв.м.)
	finishing	VARCHAR(100)	Y		Тип отделки (черновая, чистовая, дизайнерская)
	wall_material	VARCHAR(100)	Y		Материал стен (кирпич, монолит, панель)
	floor	INTEGER	Y		Этаж расположения объекта
	price	NUMERIC(15,2)	N		Основная цена объекта (руб.)
	discount_price	NUMERIC(15,2)	Y		Скидочная цена объекта (руб.)
	status	VARCHAR(50)	N	DEFAULT 'for_sale'	Статус объекта (for_sale, sold, reserved)
	address	VARCHAR(500)	Y		Адрес объекта недвижимости
	description	TEXT	Y		Описание объекта
	is_active	BOOLEAN	N	DEFAULT TRUE	Признак активности записи
	created_at	TIMESTAMP WITH TIME ZONE	Y	DEFAULT NOW()	Дата и время создания записи
	updated_at	TIMESTAMP WITH TIME ZONE	Y	DEFAULT NOW()	Дата и время последнего обновления

Таблица 7 — Таблица photos (Фотографии объектов)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор фотографии
FK	property_id	INTEGER	N		ID объекта (ссылка на properties.id)
	file_path	VARCHAR(500)	N		Путь к файлу фотографии
	description	VARCHAR(300)	Y		Описание фотографии
	created_at	TIMESTAMP WITH TIME ZONE	Y	DEFAULT NOW()	Дата и время загрузки

Таблица 8 — Таблица construction_docs (Строительная документация)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор документа
FK	property_id	INTEGER	N		ID объекта (ссылка на properties.id)
	name	VARCHAR(300)	N		Наименование документа
	doc_type	VARCHAR(50)	N		Тип документа (схема, план застройки)
	file_path	VARCHAR(500)	N		Путь к файлу документа
	created_at	TIMESTAMP WITH TIME ZONE	Y	DEFAULT NOW()	Дата и время добавления

Таблица 9 — Таблица contracts (Договоры купли-продажи)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор договора
	contract_number	VARCHAR	N	Уникальное	Номер договора
FK	property_id	INTEGER	N		ID объекта (ссылка на properties.id)
FK	created_by	INTEGER	Y		ID менеджера, оформившего договор (users.id)

	buyer_name	VARCHAR(300)	N		ФИО или наименование покупателя
	buyer_passport	VARCHAR(100)	Y		Паспортные данные покупателя
	seller_info	TEXT	Y		Сведения о продавце (застройщике)
	price	NUMERIC(15,2)	N		Цена по договору (руб.)
	contract_date	DATE	N		Дата заключения договора
	status	VARCHAR(50)	N	DEFAULT 'active'	Статус договора (active, completed, cancelled)
	notes	TEXT	Y		Дополнительные примечания к договору
	created_at	TIMESTAMP WITH TIME ZONE	Y	DEFAULT NOW()	Дата и время создания записи

Таблица 10 — Таблица audit_logs (Журнал аудита)

Ключ	Имя поля	Тип данных	Нулевые значения	Дополнительные	Описание
PK	id	SERIAL	N	Автозаполнение с шагом 1	Уникальный идентификатор записи аудита
FK	user_id	INTEGER	Y		ID пользователя, выполнившего действие (users.id)
	action	VARCHAR(50)	N		Тип выполненного действия (CREATE, UPDATE, DELETE)
	table_name	VARCHAR(50)	N		Название таблицы, в которой произошло изменение
	record_id	INTEGER	Y		ID изменённой записи
	old_values	TEXT	Y		Значения полей до изменения (JSON-формат)
	new_values	TEXT	Y		Значения полей после изменения (JSON-формат)
	ip_address	VARCHAR(45)	Y		IP-адрес пользователя
	user_agent	TEXT	Y		Информация о браузере / клиентском ПО
	created_at	TIMESTAMP WITH TIME ZONE	Y	DEFAULT NOW()	Дата и время выполнения действия

3.3. Файловая структура приложения

Файловая структура проекта организована в соответствии с принципами модульности, разделения ответственности и масштабируемости. Проект разделён на два независимых приложения: серверную часть на FastAPI (developer_api) и клиентское веб-приложение на Django (developer_django). Такое разделение позволяет независимо развивать API и пользовательский интерфейс, упрощает тестирование и сопровождение системы.

3.3.1. Структура FastAPI-приложения (developer_api)

Серверная часть системы реализована с использованием фреймворка FastAPI и организована по принципу многоуровневой архитектуры:

```
developer_api/                          # корневая директория FastAPI-
приложения
├── app/                                 # основной пакет приложения
│   ├── api/                             # маршрутизация API
│   │   ├── v1/                          # версия API v1 (базовая)
│   │   └── v2/                          # версия API v2 (расширенная,
после тестирования)
│   ├── controllers/                     # бизнес-логика приложения
│   ├── core/                            # конфигурация, настройки
безопасности
│   ├── db/                              # подключение к базе данных
(PostgreSQL)
│   ├── models/                          # ORM-модели SQLAlchemy/SQLModel
│   ├── schemas/                         # Pydantic-схемы (валидация,
сериализация)
│   ├── services/                        # сервисный слой
│   ├── __init__.py
│   └── main.py                          # точка входа FastAPI-приложения
├── tests/                               # тесты FastAPI
│   └── api/                             # API-тесты по маршрутам
│       ├── conftest.py                  # общие фикстуры для API-тестов
│       └── ...                          # тестовые файлы по модулям
├── .coverage                            # отчёт покрытия тестами
├── .env                                 # переменные окружения
├── create_role_users.py                 # скрипт создания тестовых
пользователей
└── requirements.txt                     # зависимости FastAPI-приложения
```

Таблица 11 — Назначение директорий FastAPI-приложения

Директория / файл	Назначение
app/api/v1/	Маршрутизаторы базовой версии API. Реализует основные CRUD-эндпоинты для всех сущностей системы

app/api/v2/	Расширенная версия API, сформированная по результатам тестирования v1. Содержит дополнительный функционал и исправления
app/controllers/	Бизнес-логика приложения. Контроллеры координируют взаимодействие между маршрутизаторами и сервисным слоем
app/core/	Конфигурация приложения, настройки JWT-аутентификации и механизмов безопасности
app/db/	Модуль подключения к базе данных PostgreSQL. Содержит настройки сессий SQLAlchemy/SQLModel
app/models/	ORM-модели, описывающие структуру таблиц базы данных
app/schemas/	Pydantic-схемы для валидации входящих данных и сериализации ответов API
app/services/	Сервисный слой. Содержит бизнес-правила и взаимодействует с репозиториями и моделями
app/main.py	Точка входа FastAPI-приложения. Регистрирует маршрутизаторы v1 и v2, настраивает middleware и Swagger/ReDoc
.env	Файл переменных окружения: строка подключения к PostgreSQL, секретные ключи JWT
create_role_users.py	Вспомогательный скрипт для создания тестовых пользователей с различными ролями
requirements.txt	Список зависимостей и версий библиотек FastAPI-приложения
tests/	Директория с тестовыми файлами. Содержит API-тесты и общие фикстуры (conftest.py)

3.3.2. Структура Django-приложения (developer_django)

Клиентское веб-приложение реализовано с использованием фреймворка Django по монолитной архитектуре с шаблонами, формами и административной панелью. Данные получаются от FastAPI посредством API-запросов.

```
developer_django/                    # корневая директория Django-
приложения
├── app/                             # основной Django-модуль
│   ├── assets/                      # статические файлы (CSS, JS,
изображения)
│   ├── controllers/                 # контроллеры взаимодействия с
FastAPI
│   ├── migrations/                  # миграции базы данных Django
│   ├── templates/                   # HTML-шаблоны (Django Template
Language)
│   ├── __init__.py
│   ├── admin.py                     # регистрация моделей в admin-
панели
│   ├── apps.py                      # конфигурация Django-приложения
│   ├── models.py                    # Django-модели
│   ├── tests.py                     # тесты Django
```

```
│   ├── urls.py                    # маршрутизация приложения
│   └── views.py                   # представления Django (Views)
├── django_app/                    # конфигурационный пакет проекта
│   ├── __init__.py
│   ├── asgi.py                    # ASGI-конфигурация
│   ├── settings.py                # глобальные настройки Django-
проекта
│   ├── urls.py                    # глобальная маршрутизация
проекта
│   └── wsgi.py                    # WSGI-конфигурация
├── tests/                         # тесты Django-приложения
│   ├── ui/                        # UI-тесты (пользовательские
сценарии)
│   │   └── conftest.py            # фикстуры для UI-тестов
│   └── api/                       # тесты взаимодействия с API
│       └── conftest.py            # фикстуры для API-тестов
├── .coverage
├── .coveragerc
├── db.sqlite3                     # вспомогательная БД (только для
разработки)
├── manage.py                      # точка входа Django
├── pytest.ini                     # конфигурация pytest
├── requirements.txt               # основные зависимости
└── requirements-dev.txt           # зависимости для разработки и
тестирования
```

Таблица 12 — Назначение директорий Django-приложения

Директория / файл	Назначение
app/assets/	Статические файлы: таблицы стилей CSS, клиентские скрипты JavaScript, изображения
app/controllers/	Контроллеры взаимодействия с FastAPI. Выполняют HTTP-запросы к API и передают данные в представления
app/migrations/	Автоматически генерируемые миграции Django ORM для управления схемой базы данных
app/templates/	HTML-шаблоны, формируемые на стороне сервера с использованием Django Template Language (DTL)
app/admin.py	Регистрация моделей в стандартной административной панели Django
app/models.py	Django ORM-модели, используемые для административной панели и работы с локальными данными
app/views.py	Представления Django. Обрабатывают запросы пользователя, вызывают контроллеры API и передают данные в шаблоны
app/urls.py	Маршрутизация приложения: сопоставление URL с конкретными представлениями

django_app/settings.py	Глобальные настройки Django-проекта: INSTALLED_APPS, DATABASES, TEMPLATES, MIDDLEWARE и другие
django_app/urls.py	Корневой файл маршрутизации, подключающий URL-конфигурации всех приложений проекта
tests/ui/	UI-тесты для проверки пользовательских сценариев через интерфейс веб-приложения
tests/api/	Тесты взаимодействия Django-приложения с FastAPI-бэкендом (интеграционные)
pytest.ini	Конфигурация pytest: пути к тестам, настройки Django для тестового окружения
requirements.txt / requirements-dev.txt	Файлы зависимостей. requirements-dev.txt содержит инструменты тестирования (pytest, httpx, Faker и др.)

3.3.3. Описание фикстур тестового окружения

Тестовое окружение проекта организовано с использованием механизма фикстур pytest. Файлы conftest.py размещены в каждой папке тестов и содержат повторно используемые вспомогательные объекты.

Фикстуры FastAPI (tests/api/conftest.py):

- client — тестовый HTTP-клиент (httpx.AsyncClient), настроенный для обращения к FastAPI-приложению без запуска реального сервера;
- test_db — изолированная тестовая база данных PostgreSQL, сбрасываемая перед каждым тестовым сеансом;
- auth_token — JWT-токен тестового пользователя с заданной ролью, используемый для проверки защищённых эндпоинтов;
- admin_token, manager_token, user_token — токены для проверки эндпоинтов с ролевым ограничением доступа.

Фикстуры UI-тестов (tests/ui/conftest.py):

- django_client — тестовый клиент Django (django.test.Client) для отправки HTTP-запросов к веб-приложению;
- mock_api_response — заглушка ответов FastAPI-бэкенда, используемая для изоляции UI-тестов от реального API;
- authenticated_client — тестовый клиент с предустановленной сессией авторизованного пользователя;
- test_data — набор тестовых данных (объекты недвижимости, фотографии, договоры), генерируемых с помощью библиотеки Faker.

3.4. Алгоритмы

В данном разделе описаны ключевые алгоритмы, реализованные в информационной системе АРМ застройщика. Каждый алгоритм представлен словесным описанием последовательности действий, а его графическое отображение в виде блок-схемы вынесено в Приложение Б.

3.4.1. Алгоритм авторизации и выдачи JWT (FastAPI)

Алгоритм обеспечивает аутентификацию пользователя по email и паролю с последующей выдачей пары JWT-токенов (access и refresh). Реализован в контроллере auth_controller и представлении login_view (Django). (см. Приложение Б, рисунок Б.1)

Последовательность действий:

- Пользователь открывает страницу входа и вводит email и пароль.
- Веб-приложение Django отправляет POST-запрос на эндпоинт FastAPI /api/v1/auth/login с полями email и password.
- Контроллер auth_controller.login выполняет поиск пользователя в таблице users по полю email.
- Если пользователь не найден — API возвращает статус 401 Unauthorized с сообщением «Неверный email или пароль».
- Если найден — с помощью bcrypt проверяется соответствие переданного пароля хэшу. При несовпадении возвращается 401. При успешной проверке генерируются access_token (срок жизни 30 минут) и refresh_token (срок жизни 24 часа). Токены возвращаются клиенту в формате JSON.
- Django-представление login_view сохраняет токен и данные пользователя в сессии. В зависимости от роли выполняется редирект: для admin → admin_dashboard; для manager → manager_dashboard; для user → user_home; для external → public_catalog.
- Во всех последующих запросах к защищённым эндпоинтам Django автоматически подставляет access_token в заголовок Authorization: Bearer <token>.

3.4.2. Алгоритм добавления объекта недвижимости с фотографиями (администратор)

Алгоритм описывает процесс создания записи об объекте недвижимости с последующей загрузкой фотографий администратором. Реализован в представлении admin_property_new_view и контроллере property_controller. (см. Приложение Б, рисунок Б.2)

Последовательность действий:

- Администратор авторизуется в системе (роль admin). Проверка выполняется через декоратор _resolve_admin.

- Переходит по URL /desk/admin/properties/new/.
- Представление admin_property_new_view загружает через property_type_controller список всех типов объектов для выпадающего списка.
- Пользователь заполняет форму: выбирает тип объекта, указывает площадь, этаж, материал стен, тип отделки, основную цену, скидочную цену (опционально) и адрес.
- При отправке POST-запроса вызывается property_controller.parse_property_create_post — парсинг и валидация полей. При ошибках валидации форма отображается повторно с сообщениями.
- Если данные корректны, FastAPI-сервер создаёт запись в таблице properties со статусом for_sale.
- В журнал аудита (audit_logs) автоматически добавляется запись: пользователь (администратор), действие CREATE_PROPERTY, идентификатор объекта, временная метка.
- API возвращает статус 201 Created и данные созданного объекта. Django-представление перенаправляет администратора на страницу загрузки фотографий (admin_property_photos_view).
- Администратор последовательно загружает фотографии через форму загрузки. Каждый файл проходит валидацию через Pillow (формат, размер, тип). При успешной загрузке создаётся запись в таблице photos. При ошибке выводится сообщение через messages.error.

3.4.3. Алгоритм формирования договора купли-продажи (Word/Excel)

Алгоритм позволяет менеджеру или администратору сформировать и скачать договор купли-продажи объекта недвижимости в формате Word или Excel. Реализован в представлениях contract_word_view и contract_excel_view. (см. Приложение Б, рисунок Б.3)

Последовательность действий:

- Пользователь с ролью admin или manager авторизуется в системе и переходит на страницу управления договорами /desk/admin/contracts/.
- Выбирает конкретный договор или создаёт новый, заполняя данные покупателя (ФИО, паспортные данные), выбирает объект недвижимости и указывает цену договора.
- Клиент отправляет GET-запрос на соответствующий URL: для Word — /desk/admin/contracts/{id}/word/; для Excel — /desk/admin/contracts/{id}/excel/.
- Представление вызывает contract_export_controller.export_contract для получения данных договора и объекта через FastAPI.

- FastAPI выполняет запрос к базе данных, получая данные из таблиц contracts и properties.
- Генерируется документ договора (Word или Excel) с заполнением всех реквизитов: данные сторон, характеристики объекта, цена, условия оплаты и передачи имущества.
- Django-представление оборачивает полученный контент в FileResponse, формирует имя файла (например, contract_20260516_001.docx). Браузер пользователя загружает файл.
- При ошибке (отсутствие данных, сбой API) пользователю выводится сообщение через messages.error и выполняется редирект на страницу договоров.

3.5. Документация по работе с информационной системой

В данном разделе представлена полная документация по работе с информационной системой «АРМ застройщика», включающая описание API-интерфейса, руководство пользователя веб-приложения, а также инструкцию по развёртыванию системы.

3.5.1. Документация по работе с API

API информационной системы реализован с использованием фреймворка FastAPI и поддерживает версионирование через префикс пути. Базовая версия v1 реализует полный функционал системы. Версия v2 разработана по результатам тестирования и включает дополнительные улучшения.

Версионирование реализовано через префиксы путей:

- /api/v1/ — базовая версия (первоначальная реализация с полным CRUD, аутентификацией, журналом аудита, формированием договоров);
- /api/v2/ — расширенная версия с исправлениями, выявленными в ходе тестирования: добавлена пагинация в ряде эндпоинтов, улучшена валидация входных данных, расширен функционал фильтрации объектов.

Для основных сущностей системы (пользователи, объекты недвижимости, фотографии, договоры, журнал аудита) реализованы стандартные HTTP-методы. Ниже приведён полный перечень эндпоинтов:

- GET /api/v1/properties/ — получение списка объектов недвижимости (в v2 добавлена пагинация и фильтрация по типу);
- GET /api/v1/properties/{id} — просмотр одного объекта;
- POST /api/v1/properties/ — создание нового объекта недвижимости;
- PUT /api/v1/properties/{id} — полное обновление данных объекта;
- PATCH /api/v1/properties/{id} — частичное обновление (статус, цена);

- DELETE /api/v1/properties/{id} — удаление объекта недвижимости;
- POST /api/v1/properties/{id}/photos/ — загрузка фотографии объекта;
- DELETE /api/v1/properties/{id}/photos/{photo_id} — удаление фотографии;
- POST /api/v1/properties/{id}/docs/ — добавление строительного документа;
- DELETE /api/v1/properties/{id}/docs/{doc_id} — удаление строительного документа;
- GET /api/v1/contracts/ — список договоров купли-продажи;
- POST /api/v1/contracts/ — создание договора;
- GET /api/v1/contracts/{id}/export/ — генерация договора в формате Word или Excel;
- GET /api/v1/audit/ — получение журнала аудита (с фильтрацией по пользователю и дате);
- POST /api/v1/auth/register — регистрация нового пользователя;
- POST /api/v1/auth/login — вход в систему, получение JWT-токена;
- POST /api/v1/auth/logout — выход из системы.

Аутентификация и авторизация.

В системе используется JWT-аутентификация (Bearer-токен). После успешной авторизации сервер возвращает access_token со временем жизни 30 минут. Токен передаётся в заголовке запроса: Authorization: Bearer <access_token>.

Защита эндпоинтов реализована через зависимости FastAPI:

- get_current_user() — проверка валидности токена;
- role_required(["admin"]) — доступ только для администраторов;
- role_required(["manager"]) — доступ для менеджеров;
- role_required(["admin", "manager"]) — доступ для нескольких ролей.

Для каждой версии API автоматически генерируется интерактивная документация:

- Swagger UI v1: http://localhost:8001/api/v1/docs
- ReDoc v1: http://localhost:8001/api/v1/redoc
- Swagger UI v2: http://localhost:8001/api/v2/docs
- ReDoc v2: http://localhost:8001/api/v2/redoc

Особенности реализации v1:

- Реализована полная ролевая модель доступа (admin, manager, user, external);
- Поддерживается фильтрация объектов недвижимости по типу, статусу и ценовому диапазону;
- Реализована генерация договоров в форматах Word и Excel через параметр ?format=docx или ?format=xlsx;
- Реализована регистрация и авторизация пользователей с хешированием паролей (bcrypt);

– Все записи создаются с автоматической фиксацией в журнале аудита.

Изменения в v2 по результатам тестирования:

По результатам тестирования API v1 были выявлены дефекты и зоны для улучшения. На их основании в версии v2 внесены следующие изменения:

– Добавлена пагинация для GET /properties/ и GET /contracts/ (параметры page, size) — выявлен дефект: при большом объёме данных ответ не ограничивался.
– Улучшены сообщения об ошибках валидации (422 Unprocessable Entity): добавлены понятные описания полей.
– Добавлен эндпоинт POST /api/v2/audit/export/ для экспорта журнала аудита в Excel.
– Добавлена фильтрация объектов недвижимости по нескольким параметрам одновременно.
– Исправлена ошибка рассинхронизации схемы БД и моделей в поле discount_price.

Подробное описание дефектов, обнаруженных в ходе тестирования API, представлено в Приложении А (Баг-репорт).

3.5.2. Документация по работе с веб-приложением

Веб-приложение разработано на основе фреймворка Django и функционирует как прокси-клиент к FastAPI. Все операции чтения и изменения данных выполняются через HTTP-запросы к REST API. Django-сессии используются для хранения access_token и роли текущего пользователя. При получении от API ответа со статусом 401 сессия автоматически сбрасывается, после чего пользователь перенаправляется на страницу авторизации.

Базовый URL веб-приложения: http://localhost:8000/

Таблица 13 — Тестовые учётные записи

Роль	Логин	Пароль
Администратор	admin1@example.com	password123
Менеджер	manager1@example.com	password123
Зарегистрированный пользователь	user1@example.com	password123
Незарегистрированный пользователь	—	—

1. Авторизация

Страница входа расположена по адресу GET /. Пользователь вводит логин и пароль, после чего данные передаются на POST /api/v1/auth/login. После успешной аутентификации access-токен сохраняется в Django-сессии, а пользователь перенаправляется в интерфейс,

соответствующий его роли: Администратор → /admin-panel/; Менеджер → /manager-panel/; Зарегистрированный пользователь → /user/; Незарегистрированный пользователь — доступ к публичному каталогу.

Рисунок 3 — Страница авторизации

2. Управление объектами недвижимости (администратор)

В разделе /admin-panel/properties/ администратор может просматривать список всех объектов недвижимости, выполнять поиск по типу, статусу и ценовому диапазону. Для добавления нового объекта необходимо нажать кнопку «Добавить объект» и заполнить форму с характеристиками. Создание записи выполняется через POST /api/v1/properties/. Для просмотра, изменения или удаления записи используется карточка объекта с соответствующими кнопками управления.

Рисунок 4 — Панель управления объектами недвижимости

3. Управление фотографиями объекта (администратор)

Раздел /admin-panel/properties/{id}/photos/ содержит галерею фотографий выбранного объекта. Администратор может добавить новые фотографии, удалить существующие. Загрузка выполняется через POST /api/v1/properties/{id}/photos/ с мультипартийной формой. Поддерживается загрузка нескольких фотографий одновременно.

Рисунок 5 — Управление фотографиями объекта

4. Управление строительной документацией (администратор)

Раздел /admin-panel/properties/{id}/docs/ предназначен для хранения схем и планов застройки. Администратор может добавить новый документ (схема, план), удалить устаревший. Все операции выполняются через API-методы /api/v1/properties/{id}/docs/.

5. Управление статусами и ценами (менеджер)

В разделе /manager-panel/properties/ менеджер может изменять статус объекта (for_sale → sold или reserved) и обновлять основную и скидочную цены. Смена статуса фиксируется в журнале аудита. Операция выполняется через PATCH /api/v1/properties/{id}/.

Рисунок 6 — Управление статусами объектов

6. Договоры купли-продажи (менеджер)

Раздел /manager-panel/contracts/ предназначен для ведения договоров купли-продажи. Менеджер заполняет форму: выбирает объект недвижимости, вводит данные покупателя (ФИО, паспортные данные), указывает цену и дату. После сохранения доступна кнопка «Скачать договор» в формате Word (.docx) или Excel (.xlsx).

– POST /api/v1/contracts/ — создание;
– GET /api/v1/contracts/ — получение списка;

– GET /api/v1/contracts/{id}/export/?format=docx — скачивание договора.

Рисунок 7 — Управление договорами купли-продажи

7. Журнал аудита (администратор)

Раздел /admin-panel/audit/ предназначен для просмотра журнала действий пользователей. В журнале отображаются дата и время операции, пользователь, тип действия и объект изменения. Система поддерживает фильтрацию по пользователю и диапазону дат, а также экспорт журнала в Excel через GET /api/v2/audit/export/.

Рисунок 8 — Журнал аудита

8. Публичный каталог объектов (незарегистрированные пользователи)

Незарегистрированные пользователи могут просматривать публичный каталог объектов недвижимости в разделе /public/properties/. Отображаются основная информация об объектах (тип, площадь, этаж, цена, фотографии) и строительная документация. Данные предоставляются через GET /api/v1/properties/ без использования токена авторизации.

9. Личный кабинет зарегистрированного пользователя

После авторизации зарегистрированный пользователь переходит в раздел /user/. В личном кабинете доступна информация о заключённых договорах с застройщиком. Все сведения загружаются через GET-запросы к REST API с использованием токена авторизации.

Рисунок 9 — Личный кабинет пользователя

10. Выход из системы

Для завершения работы пользователь нажимает кнопку «Выйти» в верхней части интерфейса. Выполняется POST /api/v1/auth/logout, после чего Django-сессия удаляется, а пользователь перенаправляется на страницу входа.

3.5.3. Руководство по развёртыванию проекта

В данном разделе представлено руководство по развёртыванию информационной системы «АРМ застройщика» на локальном компьютере или сервере.

Требования к системе:

– Python 3.10 и выше;
– PostgreSQL 17 и выше;
– Git (опционально);
– Современный браузер: Chrome, Firefox или Edge.

Шаг 1. Установка PostgreSQL

Необходимо скачать PostgreSQL с официального сайта https://www.postgresql.org/ и выполнить установку. После установки требуется создать базу данных:

```
CREATE DATABASE arm_developer_db;
```

Шаг 2. Клонирование репозитория

Если проект хранится в Git-репозитории, необходимо выполнить:

```
git clone <url_репозитория>
cd developer_project
```

При передаче проекта в архиве данный шаг можно пропустить.

Шаг 3. Запуск FastAPI

Перейдите в директорию API-проекта:

```
cd developer_api
```

Создайте виртуальное окружение и установите зависимости:

```
python -m venv .venv
pip install -r requirements.txt
```

После этого создайте файл .env и укажите параметры подключения к PostgreSQL и настройки JWT. (см. рис. 10)

Рисунок 10 — Пример файла .env

Для выполнения миграций используйте:

```
python app/main.py --migrate
```

Запуск API выполняется командой:

```
python app/main.py
```

После запуска будут доступны Swagger UI и ReDoc-документация.

Шаг 4. Запуск Django

Откройте новый терминал и перейдите в папку Django-проекта:

```
cd developer_django
```

Затем выполните миграции:

```
python manage.py migrate
```

Для запуска веб-приложения используйте:

```
python manage.py runserver
```

После запуска приложение будет доступно по адресу http://127.0.0.1:8000/.

Шаг 5. Проверка работоспособности

После завершения развёртывания необходимо убедиться, что FastAPI и Django-серверы успешно запущены. Далее следует открыть страницу http://127.0.0.1:8000/, выполнить вход под тестовой учётной записью администратора и проверить корректность загрузки данных. Также рекомендуется открыть Swagger UI по адресу http://127.0.0.1:8001/api/v1/docs и выполнить тестовый запрос GET /api/v1/properties/.

Примечания:

- При получении статуса 401 Django автоматически завершает сессию пользователя.
- Все уведомления об ошибках и успешных операциях выводятся через django.contrib.messages.
- Для остановки серверов используется сочетание клавиш Ctrl + C.

– При изменении адреса FastAPI необходимо обновить параметр FASTAPI_BASE_URL в .env Django-проекта.
– Полный перечень зависимостей указан в файлах requirements.txt.

3.6. Отчёт о тестировании проекта

3.6.1. Информационный заголовок

Таблица 14 — Информационный заголовок

Параметр	Значение
ID проекта	АРМ-ЗАСТРОЙЩИК-2026
Версия сборки	v1.0.0 (Django + FastAPI v1/v2)
ФИО тестировщика	Твердов Даниил Сергеевич
Тип отчёта	Финальный отчёт о тестировании
Дата составления	16 мая 2026 г.

3.6.2. Резюме

В ходе финального этапа тестирования выполнена проверка всех функциональных требований технического задания АРМ застройщика. Протестированы пользовательские сценарии четырёх ролей системы (администратор, менеджер, зарегистрированный пользователь, незарегистрированный пользователь), работа REST API на FastAPI (версии v1 и v2) и пользовательского веб-интерфейса на Django.

Тестирование охватывало два независимых приложения, входящих в состав информационной системы:

– FastAPI-бэкенд (developer_api): 48 автоматизированных тестов, покрытие кода — 82%;
– Django веб-приложение (developer_django): 155 автоматизированных тестов, покрытие кода — 84%.

Итого разработано и прогнано 203 автоматизированных тест-кейса. Все тесты завершены успешно. Критических и блокирующих дефектов не выявлено.

В ходе промежуточного тестирования API v1 было зафиксировано 4 дефекта, которые послужили основанием для разработки версии API v2 с устранёнными несоответствиями. Система соответствует требованиям технического задания и может быть рекомендована к опытной эксплуатации.

3.6.3. Область тестирования

Таблица 15 — Область тестирования

№	Модуль / файл тестов	Описание
1	test_auth_api.py	Тестирование авторизации, JWT и ролевых ограничений
2	test_properties_api.py	CRUD-операции с объектами недвижимости
3	test_photos_api.py	Загрузка и удаление фотографий объектов
4	test_contracts_api.py	Формирование и экспорт договоров купли-продажи
5	test_system_endpoints.py	Проверка health-check и системных endpoint
6	test_api_documented_defects.py	Регрессионные тесты исправленных дефектов

3.6.4. Метрики и статистика

Таблица 16 — Метрики и статистика

Показатель	FastAPI	Django
Всего тестов	48	155
Пройдено успешно	48	155
Не пройдено	0	0
Покрытие кода	82%	84%

3.6.5. Реестр дефектов

Таблица 17 — Реестр дефектов

ID	Модуль	Описание дефекта	Критичность	Статус
BUG-01	API / audit	Некорректная структура ответа журнала аудита	Средняя	Устранён в v2
BUG-02	API / properties	Неверный HTTP-статус при создании объекта (200 вместо 201)	Высокая	Устранён в v2
BUG-03	API / properties	Ошибка валидации отрицательной скидочной цены	Высокая	Устранён в v2
BUG-04	API / contracts	POST не возвращал статус 201 при создании договора	Высокая	Устранён в v2

3.6.6. Заключение

По результатам финального тестирования установлено, что информационная система «АРМ застройщика» соответствует требованиям технического задания. Все автоматизированные тесты завершены успешно. Покрытие FastAPI-бэкенда составляет 82%, покрытие Django-приложения — 84%, что превышает минимальный порог 80%, установленный планом тестирования. Все выявленные дефекты устранены и подтверждены регрессионными тестами. Система рекомендована к опытной эксплуатации.

4. ГЛОССАРИЙ

Термин	Определение
API	Интерфейс взаимодействия между различными программными компонентами.
REST	Архитектурный стиль взаимодействия web-сервисов через HTTP.
JWT	JSON Web Token — технология аутентификации и передачи данных.
Django	Python-фреймворк для разработки web-приложений.
FastAPI	Фреймворк для разработки REST API на языке Python.
PostgreSQL	Реляционная система управления базами данных.
RBAC	Модель разграничения доступа на основе ролей пользователей.
CRUD	Базовые операции с данными: создание, чтение, изменение и удаление.
ORM	Технология взаимодействия объектов приложения с базой данных.
Pydantic	Библиотека для валидации и сериализации данных.
SQLModel	ORM-библиотека для работы с базами данных на Python.
Swagger	Инструмент автоматической документации REST API.
UI	Пользовательский интерфейс приложения.
Backend	Серверная часть информационной системы.
Frontend	Клиентская часть приложения, обеспечивающая взаимодействие с пользователем.
АРМ	Автоматизированное рабочее место.
СУБД	Система управления базами данных.

5. СПИСОК ИСТОЧНИКОВ

1. Гаврилов А.В. Разработка информационных систем. — Москва: Юрайт, 2023.
2. Федорова Г.Н. Проектирование баз данных. — Санкт-Петербург: Питер, 2022.
3. Кузнецов С.Д. Основы баз данных. — Москва: Бином, 2021.
4. Марков А.С. Архитектура web-приложений. — Москва: Инфра-М, 2022.
5. Соммервилл И. Инженерия программного обеспечения. — Москва: Вильямс, 2021.
6. Липаев В.В. Программная инженерия сложных систем. — Москва: СИНТЕГ, 2020.
7. Документация Django Framework. — URL: https://docs.djangoproject.com/
8. Документация FastAPI. — URL: https://fastapi.tiangolo.com/
9. Документация PostgreSQL. — URL: https://www.postgresql.org/docs/
10. Документация Pytest. — URL: https://docs.pytest.org/
11. Документация Selenium. — URL: https://www.selenium.dev/documentation/
12. Документация SQLAlchemy. — URL: https://docs.sqlalchemy.org/
13. Документация Pydantic. — URL: https://docs.pydantic.dev/
14. Документация Alembic. — URL: https://alembic.sqlalchemy.org/
15. Документация Python 3.11. — URL: https://docs.python.org/3.11/
16. RFC 7519 JSON Web Token (JWT). — URL: https://datatracker.ietf.org/doc/html/rfc7519
17. ГОСТ 34.602-89 Техническое задание на создание автоматизированной системы.
18. ГОСТ 19.201-78 Техническое задание. Требования к содержанию и оформлению.
19. ГОСТ 7.32-2017 Отчет о научно-исследовательской работе.
20. Буч Г. Объектно-ориентированный анализ и проектирование. — Москва: Вильямс, 2020.
21. Шилдт Г. Python: полное руководство. — Москва: Диалектика, 2022.
22. Ричардсон Л. RESTful Web APIs. — O'Reilly Media, 2021.
23. python-docx: документация библиотеки. — URL: https://python-docx.readthedocs.io/
24. Pillow: документация библиотеки. — URL: https://pillow.readthedocs.io/

ЗАКЛЮЧЕНИЕ

В ходе выполнения курсового проекта была разработана информационная система «АРМ застройщика», предназначенная для автоматизации процессов хранения и учёта информации об объектах недвижимости, управления фотографиями и строительной документацией, ведения статусов объектов и оформления договоров купли-продажи. В процессе проектирования и разработки были изучены особенности предметной области застройщика, определены основные категории пользователей системы и реализованы механизмы разграничения доступа на основе ролевой модели.

В рамках проекта разработаны два взаимосвязанных приложения: REST API на базе FastAPI с поддержкой версионирования (v1 и v2) и полнофункциональное веб-приложение на Django, взаимодействующее с API посредством HTTP-запросов. Для хранения данных использована клиент-серверная система управления базами данных PostgreSQL. Архитектура проекта реализована с применением модульного подхода, обеспечивающего разделение бизнес-логики, уровня представления и слоя доступа к данным.

В ходе работы были реализованы основные функциональные возможности системы: регистрация и авторизация пользователей с использованием JWT-аутентификации, управление объектами недвижимости и их характеристиками, загрузка и удаление фотографий объектов, управление строительной документацией, формирование и экспорт договоров купли-продажи в форматах Word и Excel, ведение журнала аудита действий пользователей, а также поддержка пользовательских сценариев для различных ролей системы. Для API реализована автоматическая документация Swagger/OpenAPI.

Особое внимание было уделено тестированию программного продукта. В соответствии с требованиями методических рекомендаций разработаны unit-тесты, интеграционные тесты, API-тесты и UI-тесты на Selenium. Всего выполнен 203 автоматизированных тест-кейса. Покрытие кода FastAPI-бэкенда составило 82%, а Django-приложения — 84%, что превышает минимальные требования, установленные планом тестирования. В процессе промежуточного тестирования были выявлены и устранены дефекты, связанные с обработкой HTTP-статусов и валидацией входных данных. Корректность исправлений подтверждена регрессионными тестами.

Разработанная информационная система соответствует требованиям технического задания, обеспечивает корректную обработку данных об объектах недвижимости, устойчивую работу основных модулей и может быть рекомендована к опытной эксплуатации и дальнейшему развитию.

ПРИЛОЖЕНИЕ А

Результаты тестирования

Рисунок А.1 — Результаты запуска API-тестов v1

Рисунок А.2 — Результаты запуска и покрытие API-тестов v2

Рисунок А.3 — Результаты запуска и покрытие тестов Django

Баг-репорт по результатам тестирования API v1

ID дефекта	Описание дефекта	Шаги воспроизведения	Ожидаемый результат	Фактический результат	Критичность	Статус
BUG-01	Некорректная структура ответа журнала аудита	1. Авторизоваться как admin. 2. GET /api/v1/audit/	Ответ содержит список записей в поле data	Ответ содержит записи без обёртки в поле data	Средняя	Устранён в v2
BUG-02	Неверный HTTP-статус при создании объекта	1. Авторизоваться как admin. 2. POST /api/v1/properties/ с корректными данными	Статус ответа 201 Created	Статус ответа 200 OK	Высокая	Устранён в v2
BUG-03	Ошибка валидации отрицательной скидочной цены	1. POST /api/v1/properties/ с discount_price = -1000	Статус 422 с описанием ошибки	Объект создаётся с отрицательной ценой	Высокая	Устранён в v2
BUG-04	POST /contracts/ не возвращал статус 201	1. Авторизоваться как manager. 2. POST /api/v1/contracts/	Статус ответа 201 Created	Статус ответа 200 OK	Высокая	Устранён в v2

ПРИЛОЖЕНИЕ Б

Блок-схемы алгоритмов

Рисунок Б.1 — Алгоритм авторизации и выдачи JWT

Рисунок Б.2 — Алгоритм добавления объекта недвижимости с фотографиями

Рисунок Б.3 — Алгоритм формирования договора купли-продажи